

UNIVERSITY PROJECT — IMPLEMENTATION REPORT

Blockchain Prototype

UTXO, Proof-of-Work P2P Implementation

Akoujan Ali

International University of Rabat

April 6, 2026

<https://github.com/Sfrfl-17304/Rust-BlockChain/tree/fee-reward-implementation>

0 Contents

1	Project Overview	3
2	Learning Objectives	3
3	System Architecture	3
3.1	Data Model	3
3.2	Networking Model	3
4	Core Algorithms	4
4.1	Proof-of-Work	4
4.2	UTXO Validation	4
4.3	Chain Validation	4
5	Consensus and Synchronization	4
6	Algorithms in Pseudocode	5
6.1	Proof-of-Work Mining	5
6.2	Transaction Validation with UTXO	5
6.3	Block Acceptance	5
6.4	Peer Synchronization	6
7	Execution Examples	6
7.1	Single Miner, One Receiver	6
7.2	Tamper Detection	6
8	Screenshots	7
9	Discussion and Limitations	7
10	Conclusion	7

1 Project Overview

This report presents a minimal blockchain prototype implemented in Rust. In my understanding, the project is best viewed as a controlled environment to study core algorithms: UTXO-based accounting, proof-of-work, and basic peer-to-peer synchronization. The design intentionally prioritizes clarity over production security or optimization.

The system runs multiple local nodes (one per port). Each node exchanges blocks and transactions, maintains a local UTXO set, and performs validation to detect inconsistencies or corrupted data. The primary goal is to show how the core mechanics of a Bitcoin-like system can be implemented and reasoned about.

2 Learning Objectives

The following objectives guided the implementation and evaluation:

- Understand how UTXOs encode ownership and spending rules.
- Implement proof-of-work as a practical anti-tampering mechanism.
- Validate block and transaction integrity end-to-end.
- Observe simple peer synchronization under conflicting states.
- Build a minimal system that remains understandable to a student reader.

3 System Architecture

3.1 Data Model

- **Transactions** follow the UTXO model: inputs reference previous unspent outputs, and outputs define new ownership.
- **Blocks** include a header (index, previous hash, timestamp, nonce) and a list of transactions. The block hash is computed over all fields.
- **UTXO Set** is a key-value store mapping output references to spendable outputs. It is rebuilt and updated as blocks are applied.

3.2 Networking Model

Nodes communicate over TCP on 127.0.0.1. Each node:

- Listens for incoming blocks and transactions.
- Broadcasts newly mined blocks to peers.
- Requests peer chains for synchronization when conflicts appear.

The networking layer is intentionally minimal to highlight the algorithmic flow. Messages are serialized as compact strings to keep dependencies light.

4 Core Algorithms

4.1 Proof-of-Work

In my implementation, mining is a search problem over a nonce. A block is accepted only if its hash starts with a fixed number of leading zeros. This introduces computational cost and makes it difficult to alter history without re-mining.

4.2 UTXO Validation

I enforce the following rules for each non-coinbase transaction:

1. Every input must reference an existing unspent output.
2. No input may be reused within the same block (double-spend prevention).
3. The total value of inputs must be at least the total value of outputs.

Coinbase transactions are restricted to exactly one per block and must appear first, which defines how new coins enter the system.

4.3 Chain Validation

A chain is considered valid when:

- Each block hash equals the recomputed hash of its contents.
- Each block links correctly to the previous block hash.
- Each block meets the difficulty target.
- The UTXO set remains consistent while replaying the chain.

5 Consensus and Synchronization

The consensus rule used is the longest valid chain. When a node receives a block that cannot be attached, it requests peers' chains and adopts the best valid candidate. This

mechanism is a simplified version of Nakamoto consensus and is sufficient for illustrating how agreement can emerge in a distributed setting.

6 Algorithms in Pseudocode

6.1 Proof-of-Work Mining

```
function MINE_BLOCK(block, difficulty):
    target_prefix = "0" repeated difficulty times
    while true:
        hash = HASH(block.header + block.transactions + block.nonce)
        if hash starts with target_prefix:
            block.hash = hash
            return block
        block.nonce = block.nonce + 1
```

6.2 Transaction Validation with UTXO

```
function VALIDATE_AND_APPLY(transactions, utxos):
    if transactions is empty: return false
    if first transaction has inputs: return false
    for each tx after first:
        if tx.inputs is empty: return false

    spent_in_block = empty set
    for each tx in transactions:
        if tx is coinbase:
            add its outputs to utxos
            continue
        sum_in = 0; sum_out = 0
        for each input in tx.inputs:
            if input in spent_in_block: return false
            if input not in utxos: return false
            sum_in += utxos[input].amount
            mark input as spent_in_block
        for each output in tx.outputs:
            sum_out += output.amount
        if sum_in < sum_out: return false
        remove inputs from utxos; add outputs to utxos
    return true
```

6.3 Block Acceptance

```
function ACCEPT_BLOCK(block, chain):
    if block.index != chain.length: return false
    if block.prev_hash != chain.tip.hash: return false
    if block.hash != HASH(block.contents): return false
```

```
if not POW_OK(block.hash, difficulty): return false
if not VALIDATE_AND_APPLY(block.txs, chain.utxos): return false
append block to chain
return true
```

6.4 Peer Synchronization

```
function SYNC_FROM_PEERS(peers):
    best_chain = local_chain
    for each peer in peers:
        candidate = request_chain(peer)
        if candidate is valid and longer than best_chain:
            best_chain = candidate
    local_chain = best_chain
```

7 Execution Examples ---

7.1 Single Miner, One Receiver

```
# Node 1 (miner)
cargo run -- 7001 127.0.0.1:7002 --miner
> mine
> send miner-7002 10
> mine

# Node 2 (receiver)
cargo run -- 7002 127.0.0.1:7001
> balance
```

7.2 Tamper Detection

```
> tamper
> chain
# Expected: chain validity becomes false
```

8 Screenshots

```
Node miner-7001 running.
Commands: send <to> <amount> | mine | balance | chain | tamper
Note: mining requires --miner
> mine
> block mined and broadcast
received block (index=1, prev=1b6922c4...)
-> rejected (current height=2, tip=002256fd...)
balance
balance: 50
> send miner-7002 10
```

Figure 1: Node 1 mining and broadcasting blocks.

```
mining disabled (start with --miner)
> received tx
balance
balance: 0
> received block (index=2, prev=002256fd...)
-> accepted (extended chain)
balance
balance: 10
> send miner-7001 5
tx broadcast
```

Figure 2: Node 2 receiving and validating blocks from Node 1.

```
balance: 10
> send miner-7001 5
tx broadcast
> mine
mining disabled (start with --miner)
> received block (index=3, prev=009983be...)
-> accepted (extended chain)
balance
balance: 5
>
```

Figure 3: UTXO balance changes after transactions are mined.

9 Discussion and Limitations

While the prototype captures the core ideas, it omits several real-world requirements. There are no digital signatures, no mempool eviction policy, no transaction fees, and no persistent storage. Networking is simplified and does not implement peer discovery or adversarial behavior. These omissions were intentional to keep the system readable and suitable for learning.

10 Conclusion

This prototype helped me understand the algorithmic foundations of a Bitcoin-like blockchain: how UTXOs represent ownership, why proof-of-work protects history, and how basic synchronization allows nodes to converge on a common ledger. The implementation is intentionally minimal, which makes the core ideas explicit and easier to reason about.

Future work could include persistent storage, digital signatures, improved serialization, and more robust peer discovery.